

Experiment:36

With linked allocation, each file is a linked list of disk blocks;the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file.Each block contains a pointer to the next block. Design a C program to simulate the file allocation strategy.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct Block
{
    char data[256];
    struct Block *next;
};

int main()
{
    struct Block *firstBlock = NULL;
    struct Block *lastBlock = NULL;
    int blockCount = 0;
    int blockNumber;
    char data[256];
    char choice;

    printf("Linked Allocation Simulation\n");

    while (1)
```

```
{
    printf("\nEnter 'W' to write a block, ");
    printf("'R' to read a block, or 'Q' to quit: ");
    scanf(" %c", &choice);
    if (choice == 'Q' || choice == 'q')
    {
        break;
    }
    if (choice == 'W' || choice == 'w')
    {
        printf("Enter data for the block: ");
        scanf(" %[^\n]", data);

        struct Block *newBlock =
            (struct Block *)malloc(sizeof(struct Block));
        if (newBlock == NULL)
        {
            printf("Memory allocation failed.\n");
            return 1;
        }
        strcpy(newBlock->data, data);
        newBlock->next = NULL;

        if (blockCount == 0)
        {
            firstBlock = newBlock;
```

```
        lastBlock = newBlock;
    }
    else
    {
        lastBlock->next = newBlock;
        lastBlock = newBlock;
    }
    blockCount++;
    printf("Block %d allocated successfully.\n", blockCount);
}
else if (choice == 'R' || choice == 'r')
{
    if (blockCount == 0)
    {
        printf("No blocks available.\n");
        continue;
    }
    printf("Enter the block number to read (1-%d): ",
        blockCount);
    scanf("%d", &blockNumber);

    if (blockNumber < 1 || blockNumber > blockCount)
    {
        printf("Invalid block number.\n");
    }
    else
```

```
{
    struct Block *currentBlock = firstBlock;
    for (int i = 1; i < blockNumber; i++)
    {
        currentBlock = currentBlock->next;
    }
    printf("Block %d Data: %s\n",
        blockNumber, currentBlock->data);
}

else
{
    printf("Invalid choice.\n");
}

struct Block *currentBlock = firstBlock;
while (currentBlock != NULL)
{
    struct Block *nextBlock = currentBlock->next;
    free(currentBlock);
    currentBlock = nextBlock;
}

return 0;
}
```

Output:

```
Linked Allocation Simulation

Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: w
Enter data for the block: w
Block 1 allocated successfully.

Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: r
Enter the block number to read (1-1): q
Invalid block number.

Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit:

...Program finished with exit code 0
Press ENTER to exit console.[]
```